

Funktion `isolate_finger(...)`

1 Anforderungen

- Abschneiden des verletzten Fingers vom 3D-Scan
- Möglichst wenig Userinput, da dies zu einer hohen Fehlerquote führt
- Schnitt auf Höhe der Fingerwurzel (also das Metakarpalgelenk)
- Schnitt soll bei allen Untersuchungen am möglichst gleichen Ort passieren, also deterministisch
- Danach deterministisches Ausrichten des Finger, möglichst senkrecht

2 Grundgedanken

Am Besten wäre natürlich, den Schnittprozess komplett ohne Userinput entwickeln zu können. Das scheitert aber an zwei Problemen:

1. Wahl des korrekten Fingers
2. Haltung der Finger und Qualität des Scans

Falls aber in Zukunft mit der neuen 3D-Kamera bzw. dem „Kameratunnel“ bessere Scans inklusive besserer Markierungen produziert werden können, kann Problem 1 gelöst werden, indem der Algorithmus den dann einzigen Finger, welcher einige Faces (Threshold der Anzahl muss getestet werden) in der Farbe der Markierung auswählt. Problem 2 kann folgend gelöst werden: Die Patientin oder der Patient soll ermutigt werden, die Hand beim Aufnehmen des Scans möglichst weit und gerade auszustrecken. Zuerst Ausrichtung der Hand, dass sie „aufrecht“ steht. Das kann gelöst werden, indem der Algorithmus die Hand auf die xz-Ebene projiziert. Dann wird die Hand um jeweils 1° gedreht, und die Stellung, welche die kleinste Fläche erzeugt, wird ausgewählt. Die Hand steht nun aufrecht (oder genau „falsch“ herum, das soll dann mit Fehlerbehandlung gefixt werden). Dann schneidet der Algorithmus die Hand in viele kleine Flächen. Die Schnittebenen sind dann mit der y-Achse als Normalenvektor definiert. Die Schnittbilder ähneln dann auf Höhe der 4 Finger (also vor dem Mittelhand) 4 Kreisen oder Ellipsen. Falls dann die Ellipse des verletzten Fingers schließlich mit der des Nachbarfingers „zusammenwächst“ kann diese Ebene als Höhe h festgelegt werden. Daraufhin folgt der Algorithmus dem der etwas weniger qualitativen Scans, siehe dazu unten (1).

Hinweis zum aktuellen Implementierungsstand: Der tatsächlich implementierte Ausrichtungsschritt (Funktion `rotationmatrix/richte_hand_aus`, siehe Abschnitt 4) verwendet nicht die hier beschriebene Flächen-Minimierung, sondern eine robustere, auf den beiden angeklickten Punkten basierende Methode („gleiche Höhe“-Ausrichtung), da diese ohne die Unsicherheiten einer indirekten Flächen-Heuristik auskommt.

3 Algorithmus – für aktuelle Scans

- Lade den Scan mit dem Package `pyvista`
- User markiert den Fingernagel des verletzten Fingers
 - Präzision ist dabei unwichtig, da folgender User-Error nahezu nicht in das Endergebnis mit einfließt
- User wird aufgefordert, den Fingernagel des benachbarten Fingers auszuwählen, zur Konsistenz:

Verletzter Finger	Wähle aus
Zeigefinger	Mittelfinger
Mittelfinger	Ringfinger
Ringfinger	kl. Finger
kl. Finger	Ringfinger

- Danach wird mit Dijkstras Algorithmus der kürzeste Weg der beiden Punkte berechnet.
Sei $G = (V, E)$ der Graph aus Mesh-Vertices V und Kanten E (mit Kantenlänge als Gewicht $w(u, v)$), p_1 der Vertex des verletzten Fingers und p_2 der Vertex des Nachbarfingers:

Algorithm 1 Dijkstra – kürzester Pfad zwischen p_1 und p_2

```

1:  $dist[p_1] \leftarrow 0$ ;  $dist[v] \leftarrow \infty$  für alle  $v \neq p_1$ 
2:  $Q \leftarrow$  Prioritätswarteschlange mit allen  $v \in V$ , priorisiert nach  $dist[v]$ 
3: while  $Q$  nicht leer do
4:    $u \leftarrow \text{ExtractMin}(Q)$ 
5:   if  $u = p_2$  then
6:     break
7:   end if
8:   for jeder Nachbar  $v$  von  $u$  do
9:     if  $dist[u] + w(u, v) < dist[v]$  then
10:       $dist[v] \leftarrow dist[u] + w(u, v)$ 
11:       $vorgaenger[v] \leftarrow u$ 
12:       $\text{DecreaseKey}(Q, v, dist[v])$ 
13:    end if
14:  end for
15: end while
16: Pfad rekonstruieren durch Rückverfolgen von  $vorgaenger$  ab  $p_2$  bis  $p_1$ 

```

In der Implementierung übernimmt `pyvistas` eingebaute Funktion `mesh.geodesic(idx_a, idx_b)` (intern VTKs `vtkDijkstraGraphGeodesicPath`) diese Berechnung.

- (1) Die Höhe des niedrigsten (nach z) Vertex wird als Höhe h gespeichert:

$$h = \min_{v \in \text{Pfad}} v_z$$

- Normale der finalen Schnittfläche wird berechnet:
 - Der vom User ausgewählte Punkt „wächst“ nach dem Prinzip der Breitensuche um eine gesetzte Anzahl an Vertices (Anzahl nach Testen: 3000)

- Berechnen einer Achse s der Punkte mit Hilfe von Principal Component Analysis:
Sei $\{x_i\}_{i=1}^N$ die Menge der N nächstgelegenen Vertices um den angeklickten Punkt.
Mittelwert:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

Kovarianzmatrix:

$$\Sigma = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})(x_i - \bar{x})^\top$$

Eigenwertzerlegung $\Sigma v_j = \lambda_j v_j$; die Achse s ist der Eigenvektor zum größten Eigenwert:

$$s = v_{j^*}, \quad j^* = \arg \max_j \lambda_j$$

Vorzeichen-Korrektur (Achse soll vom Hand-Schwerpunkt c weg zeigen):

$$s \leftarrow \begin{cases} -s & \text{falls } s \cdot (x_{\text{Klick}} - c) < 0 \\ s & \text{sonst} \end{cases}$$

- s folgt jetzt nahezu exakt als „Achse des Fingers“
- s wird als Normale der Ebene definiert
- Jetzt wird der nächste Punkt der Achse s mit dem tiefsten Punkt des Dijkstra-Weges h gesucht.

Die Achse s verläuft als Gerade durch das lokale Zentrum z_{lok} (Mittelwert der PCA-Nachbarschaft):

$$P(t) = z_{\text{lok}} + t \cdot s$$

Gesucht ist t_{boden} , sodass $P(t)_z = h$:

$$t_{\text{boden}} = \frac{h - (z_{\text{lok}})_z}{s_z}$$

- Rotationsellipsoid wird berechnet, welches als Rotationsachse Achse s besitzt. Die Länge der Achse ist 1,6-mal die Länge des nächsten Punktes der Achse s zu h bis zum weit entfernten Punkt des Fingers zu h .

Für jeden Punkt x_i der PCA-Nachbarschaft wird die axiale Projektion und der radiale Abstand zur Achse berechnet:

$$t_i = (x_i - z_{\text{lok}}) \cdot s, \quad r_i = \|(x_i - z_{\text{lok}}) - t_i s\|$$

Äquator-Radius (doppelter größter radialer Abstand):

$$b = 2 \cdot \max_i r_i$$

Obere Grenze und Original-Länge:

$$t_{\text{oben}} = \max_i t_i, \quad L = t_{\text{oben}} - t_{\text{boden}}$$

Halbe Hauptachse (Gesamtlänge des Ellipsoids somit $1,6 \cdot L$):

$$a = 0,8 \cdot L$$

Untere Spitze liegt um den Anteil **unterschreitung** unterhalb von t_{boden} :

$$t_{\text{unten}} = t_{\text{boden}} - \text{unterschreitung} \cdot L, \quad t_{\text{zentrum}} = t_{\text{unten}} + a$$

Mittelpunkt des Ellipsoids:

$$m_{\text{ellipsoid}} = z_{\text{lok}} + t_{\text{zentrum}} \cdot s$$

Das Rotationsellipsoid selbst ist in seinem lokalen, auf $m_{\text{ellipsoid}}$ zentrierten und auf s ausgerichteten Koordinatensystem (x, y, z) definiert durch:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{b^2} = 1$$

Die Ausrichtung von der Standard- x -Achse auf s erfolgt über die Rodrigues-Rotationsformel (siehe Abschnitt 4, `rotationsmatrix_a_nach_b`).

- Schneiden, neuen 3D-Scan laden und speichern, dabei soll ein Geschwisterordner namens „isolierte_scans“ zu „originale_scans“ erstellt werden. Darin soll ein Ordner erstellt werden, welcher als Name den Originalnamen des Originalscans + „_isoliert“ hat. Darin werden alle veränderten und geschnittenen Daten (obj, jpg's, mtl) gespeichert.

4 Teilfunktionen-Referenz

Die folgende Liste enthält alle im Code enthaltenen Teilfunktionen, inklusive derer, die in der ursprünglichen Beschreibung nicht explizit genannt waren.

- `load_teilmeshe_mit_textur(obj_pfad)` – Lädt die OBJ-Datei über `trimesh` (zuverlässig auch bei mehreren Materialien/Texturen) und wandelt jedes Teilmesh in ein `pyvista.PolyData` mit zugehöriger `pv.Texture` um. Teilmeshe ohne UV-Koordinaten werden übersprungen.
- `zeige_mit_texturen(plotter, teile)` – Zeigt mehrere texturierte Teile in einem Plotter an, da `add_mesh(..., texture=...)` pro Aufruf nur eine Textur unterstützt.
- `transformiere_teile(teile, transform_matrix)` – Wendet dieselbe 4×4 -Transformationsmatrix auf jedes einzelne Teil an, damit alle Teile synchron zum gemergten Haupt-Mesh bleiben.
- `clippe_teile(teile, form)` – Wendet dieselbe Schnittform (Ellipsoid) auf jedes einzelne Teil an; leer geschnittene Teile werden verworfen.
- `pick_finger_point(plotter, mesh)` – Interaktives Auswählen der beiden Punkte (letzter Finger, Nachbarfinger) in einem durchgehenden Fenster.
- `rotationsmatrix(seed, second_finger, centroid)` – Berechnet die Rotationsmatrix R , die `seed` und `second_finger` auf exakt dieselbe Höhe (z) bringt und ihren Mittelpunkt dabei maximal hoch platziert. Sei $v = p_2 - p_1$, $\hat{v} = v/\|v\|$ und $m = \frac{p_1 + p_2}{2} - c$ (mit Schwerpunkt c). Zerlegung von m :

$$m_{\parallel} = (m \cdot \hat{v}) \hat{v}, \quad m_{\perp} = m - m_{\parallel}, \quad \hat{m}_{\perp} = \frac{m_{\perp}}{\|m_{\perp}\|}$$

Lokale und Ziel-Basis:

$$B_{\text{lok}} = (\hat{v} \quad \hat{m}_{\perp} \quad \hat{v} \times \hat{m}_{\perp}), \quad B_{\text{ziel}} = (e_x \quad e_z \quad e_x \times e_z)$$

$$R = B_{\text{ziel}} B_{\text{lok}}^{\top}$$

- `richte_hand_aus(mesh, verletzter_finger, second_finger)` – Wendet R aus `rotationsmatrix` auf das gesamte Mesh sowie die beiden Punkte an (Rotation um den Schwerpunkt):

$$T = \begin{pmatrix} R & -Rc \\ 0 & 1 \end{pmatrix}$$

- `dijkstra_und_tiefster_punkt(mesh, verletzter_finger, second_finger)` – Siehe Dijkstra-Pseudocode oben; zusätzlich wird eine kleine Kugel zur visuellen Markierung des tiefsten Punktes erzeugt.
- `rotationsmatrix_a_nach_b(a, b)` – Allgemeine Rodrigues-Rotationsformel, dreht einen beliebigen Vektor a auf einen beliebigen Vektor b (wird zur Ausrichtung des Ellipsoids benötigt). Mit $\hat{a}, \hat{b}$ normiert, Rotationsachse $k = \hat{a} \times \hat{b}$, $\sin \theta = \|k\|$, $\cos \theta = \hat{a} \cdot \hat{b}$ und Kreuzprodukt-Matrix K der normierten Achse:

$$R = I + K \sin \theta + K^2(1 - \cos \theta)$$

Sonderfälle für $\sin \theta \rightarrow 0$ (parallele bzw. exakt entgegengesetzte Vektoren) werden gesondert behandelt.

- `finger_normale(mesh, verletzter_finger, anzahl_punkte)` – PCA-Berechnung der Finger-Achse, siehe Formeln oben.
- `erstelle_schnitt_ellipsoid(...)` – Berechnung des Rotationsellipsoids, siehe Formeln oben.
- `speichere_isolierten_finger(scan_ordner_pfad, texture_teile_isoliert)` – Erstellt die Ordnerstruktur (`isolierte_scans/<name>_isoliert/`) und exportiert alle isolierten, texturierten Teile als gemeinsames Multi-Material-OBJ (inkl. MTL und Textur-Bildern) über eine `trimesh.Scene`.

5 Aufruf und Rückgabe

Aufruf:

```
isolate_finger(path: str)
```

Rückgabe:

Keine. Speicherung des isolierten Fingers siehe letzter Punkt des Algorithmus.

6 Aktueller Programmierungsstatus

Abgeschlossen. Ich würde die Funktion jedoch mit neuen Testdaten des „Kameratunnels“ anpassen und verbessern.